



BİL211

Computer Programming II

Dr. Bahaeddin ERAVCI
beravci@etu.edu.tr

Class Review

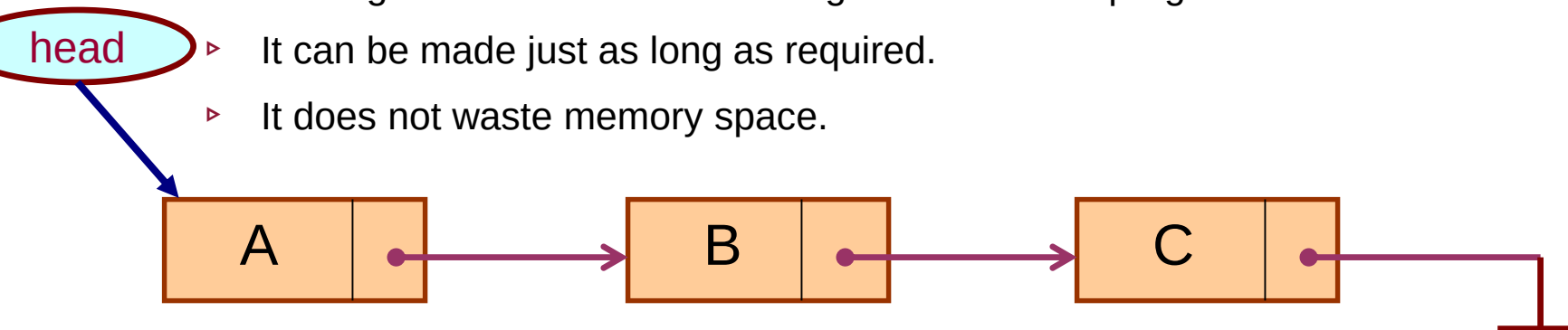
Linked Data Introduction



Types

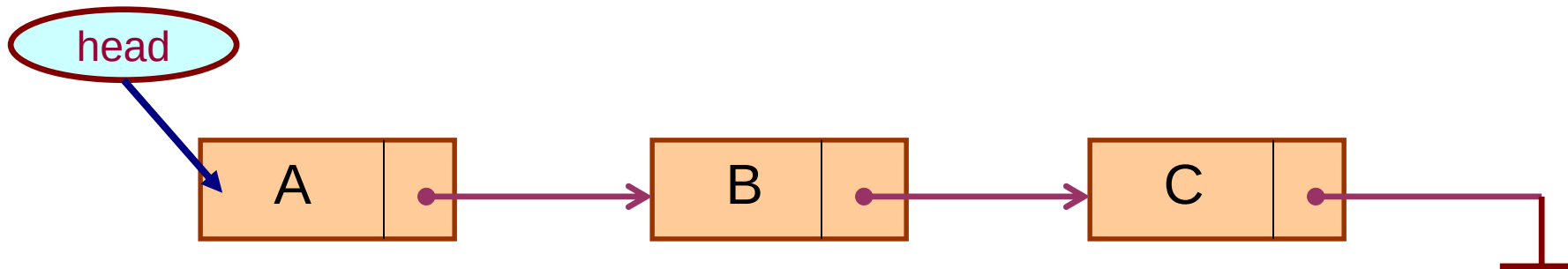
Linked Data

- ▶ Linked data structure can change during execution
 - ▶ Successive elements are connected by pointers.
 - ▶ Last element points to NULL.
 - ▶ It can grow or shrink in size during execution of a program.
 - ▶ It can be made just as long as required.
 - ▶ It does not waste memory space.



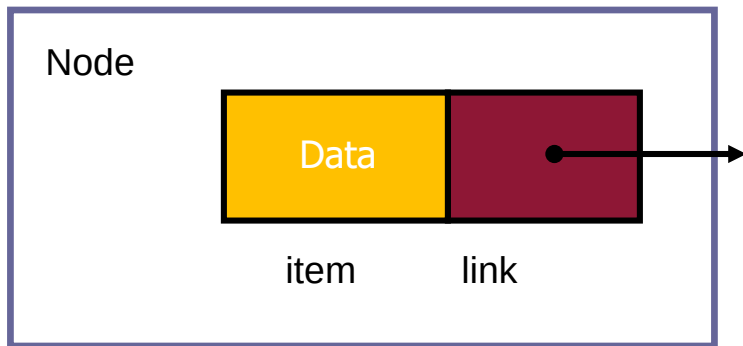
Linked Data

- ▶ Keeping track of a linked data:
 - ▶ Must know the pointer to the first element of the structure (called start, head, root etc.).



Linked Data

- ▶ Linked Data Needs Nodes
- ▶ Can be inner class, doesn't have to



```
private class Node {
    private String item;
    private Node link;

    public Node() {
        item = null;
        link = null;
    }

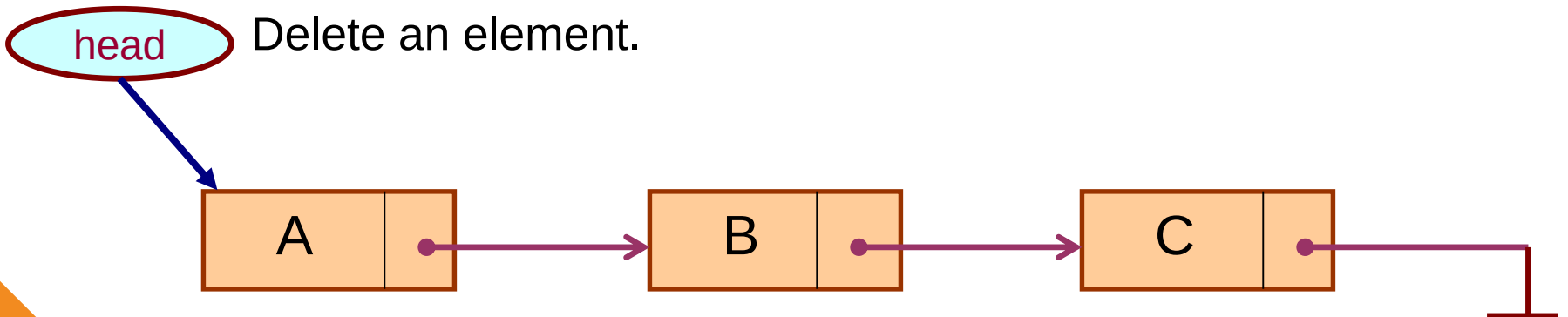
    public Node(String newItem, Node linkValue) {
        item = newItem;
        link = linkValue;
    }
}

} // End of Node inner class
```

Linked List

- ▶ A linked list is a series of connected nodes
- ▶ Linked lists provide flexibility in allowing the items to be rearranged efficiently.
 - ▶ Insert an element.

Delete an element.



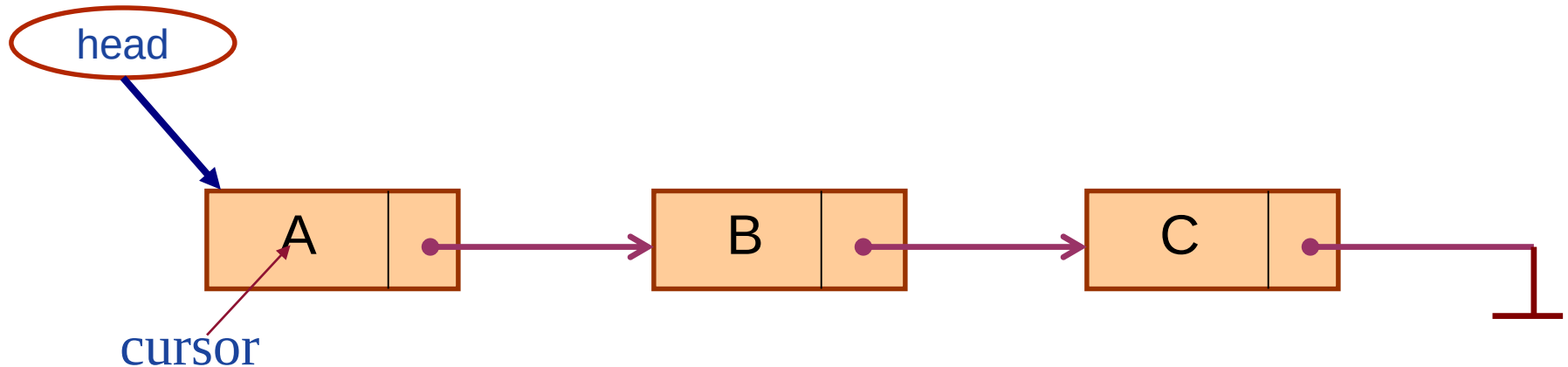
Linked List

- ▶ Keep track of head!
- ▶ At the beginning null

```
public class LinkedList {  
    private class Node { ...  
  
    private Node head;  
  
    public LinkedList() {  
        head = null;  
    }  
}
```

Linked List - Size

- ▶ Traversing a linked list

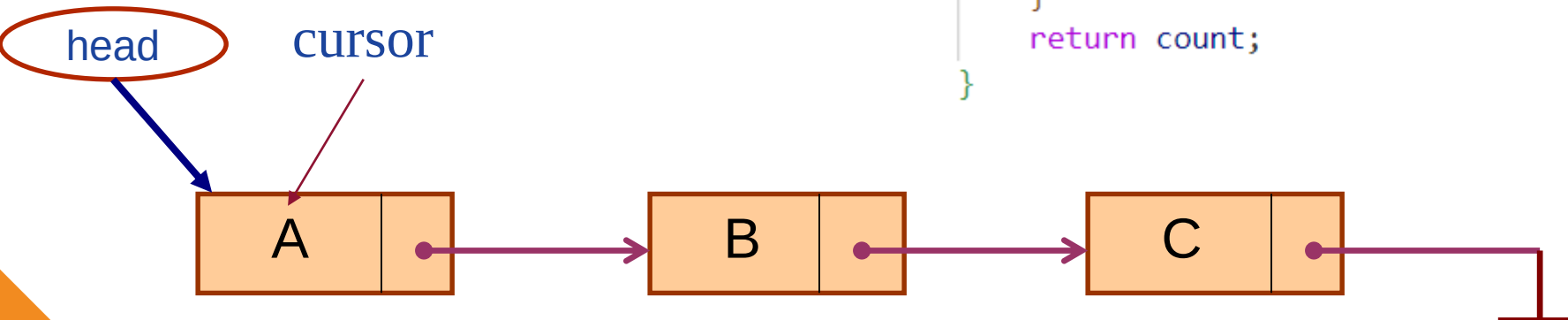


Linked List - Size

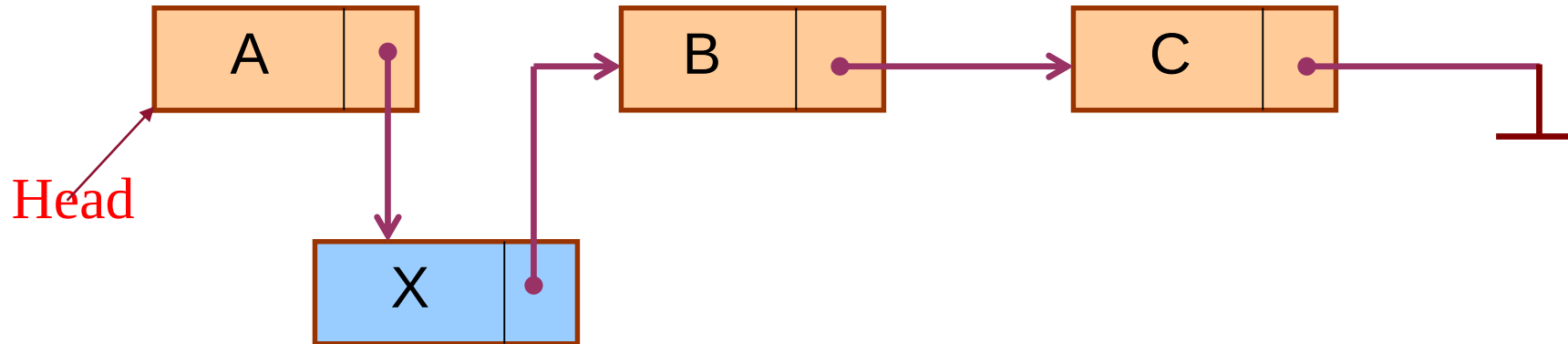
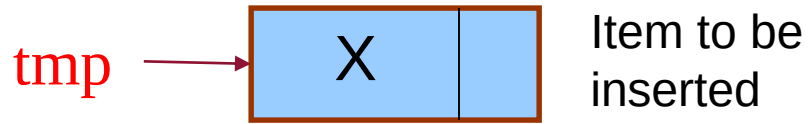
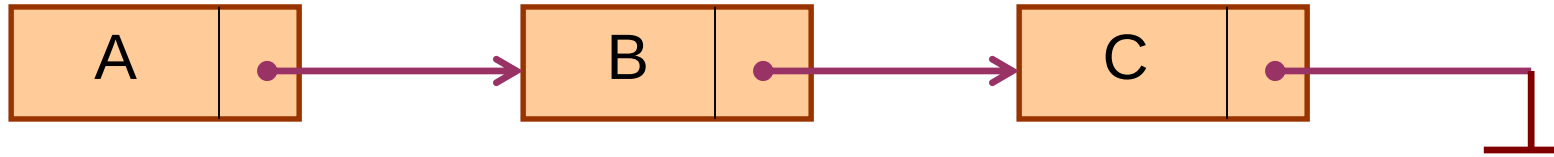
- Traversing a linked list

```
public int sizeRecursive(Node head) {  
    if (head == null)  
        return 0;  
    else  
        return 1 + sizeRecursive(head.link);  
}
```

```
public int size() {  
    int count = 0;  
    Node cursor = head;  
    while (cursor != null) {  
        count++;  
        cursor = cursor.link;  
    }  
    return count;  
}
```



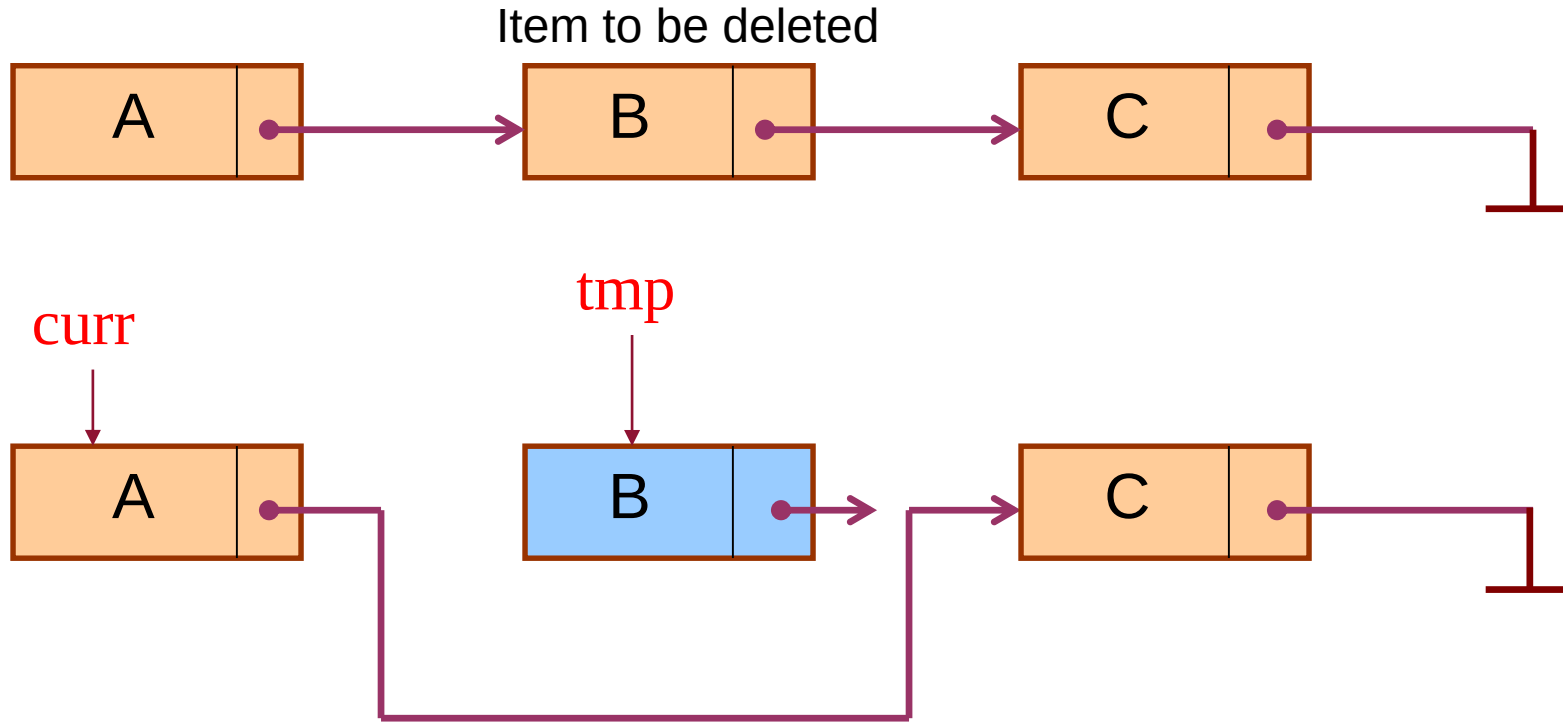
Linked List - Add



Linked List - Add

```
public void addToStart(String itemName) {  
    head = new Node(itemName, linkValue:null);  
}  
  
public void add(String itemName, int index) {  
    if (index == 0) {  
        addToStart(itemName);  
    } else {  
        Node position = head;  
        for (int i = 0; i < index; i++) {  
            if (position.link == null) {  
                throw new IndexOutOfBoundsException(s:"Index out of bounds");  
            }  
            position = position.link;  
        }  
  
        Node newNode = new Node(itemName, position.link);  
        position.link = newNode;  
    }  
}
```

Linked List - Remove



Linked List - Remove

```
public boolean remove(int index) {  
    if (index == 0) {  
        return deleteHeadNode();  
    } else {  
        Node position = head;  
        for (int i = 0; i < index; i++) {  
            if (position.link == null) {  
                throw new IndexOutOfBoundsException(s:"Index out of bounds");  
            }  
            position = position.link;  
        }  
        position.link = position.link.link;  
        return true;  
    }  
}
```

```
public boolean deleteHeadNode() {  
    if (head != null) {  
        head = head.link;  
        return true;  
    } else  
        return false;  
}
```

Linked List - Other

```
private Node find(String target) {  
    Node position = head;  
    String itemAtPosition;  
    while (position != null) {  
        itemAtPosition = position.item;  
        if (itemAtPosition.equals(target))  
            return position;  
        position = position.link;  
    }  
    return null; // target was not found  
}
```

```
public boolean isEmpty() {  
    return (head == null);  
}
```

```
public void clear() {  
    head = null;  
}
```



```
public class LinkedListGeneric<T> {
    private class Node<T> {
        private T data;
        private Node<T> link;

        public Node() {
            data = null;
            link = null;
        }

        public Node(T newData, Node<T> linkValue) {
            data = newData;
            link = linkValue;
        }
    }
} // End of Node<T> inner class

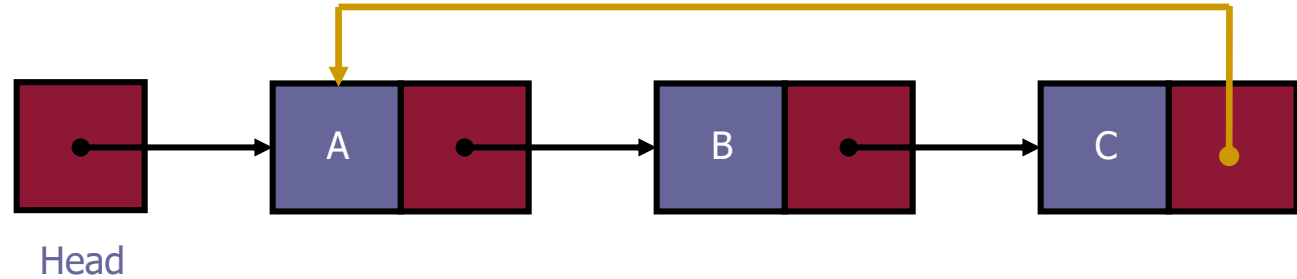
private Node<T> head;
```

Linked List - Generic

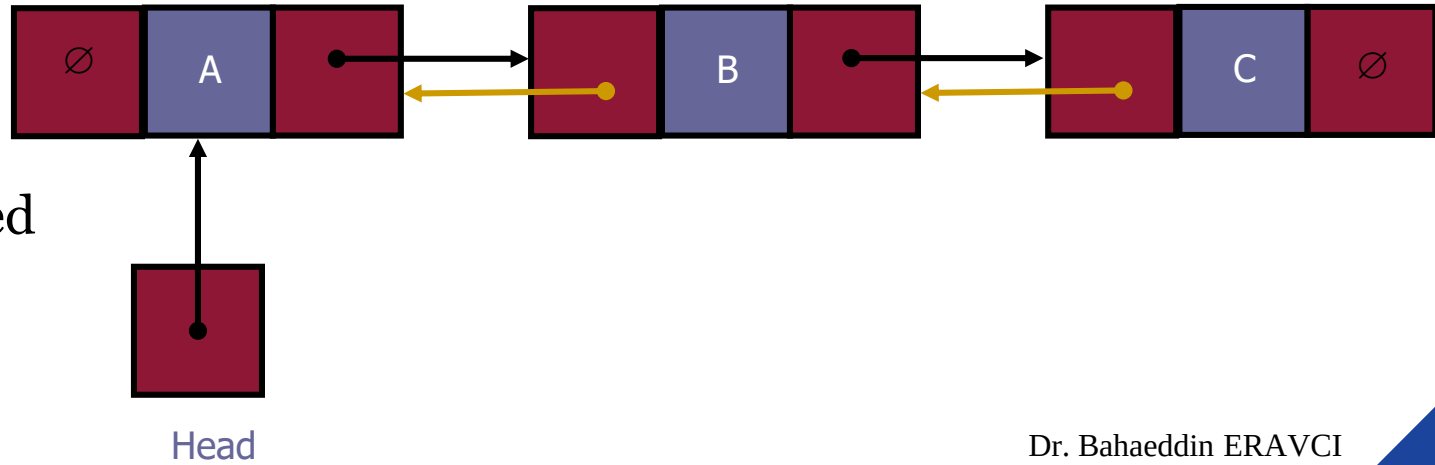
```
LinkedListGeneric<String> list = new LinkedListGeneric<String>();  
list.addToStart(itemData:"Hello");  
list.addToStart(itemData:"World");  
list.addToStart(itemData:"Java");  
list.addToIndex(itemData:"C", index:1);  
list.outputList();  
  
LinkedListGeneric<Integer> list2 = new LinkedListGeneric<Integer>();  
list2.addToStart(itemData:1);  
list2.addToStart(itemData:2);  
list2.addToStart(itemData:3);  
list2.addToIndex(itemData:4, index:1);  
list2.outputList();  
System.out.println("Size: " + list2.size());  
System.out.println("Is empty: " + list2.isEmpty());
```


Linked List - Variations

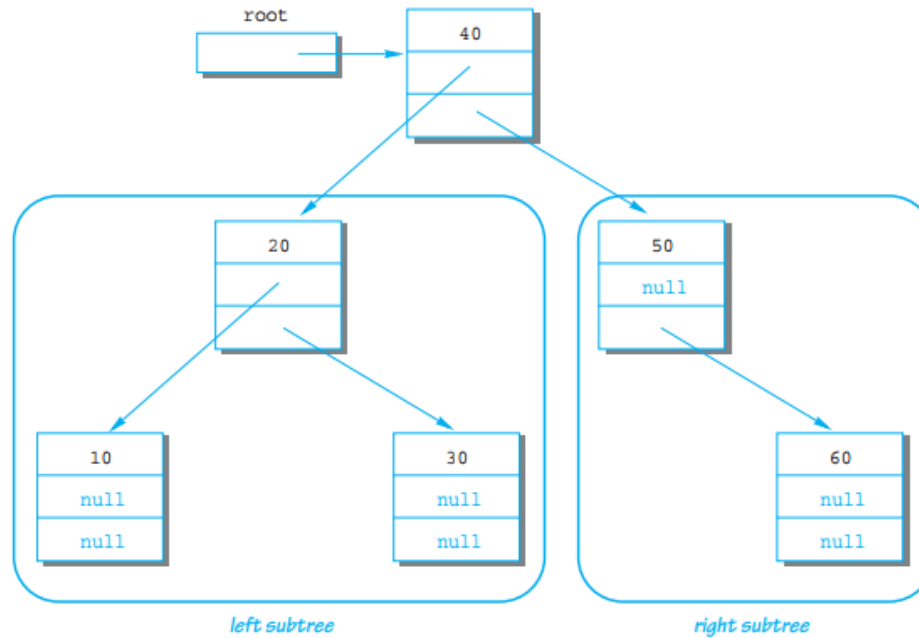
► Circular:



► Doubly Linked



Trees

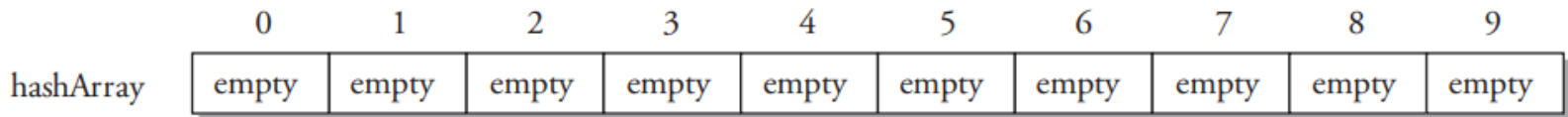


Hash Map

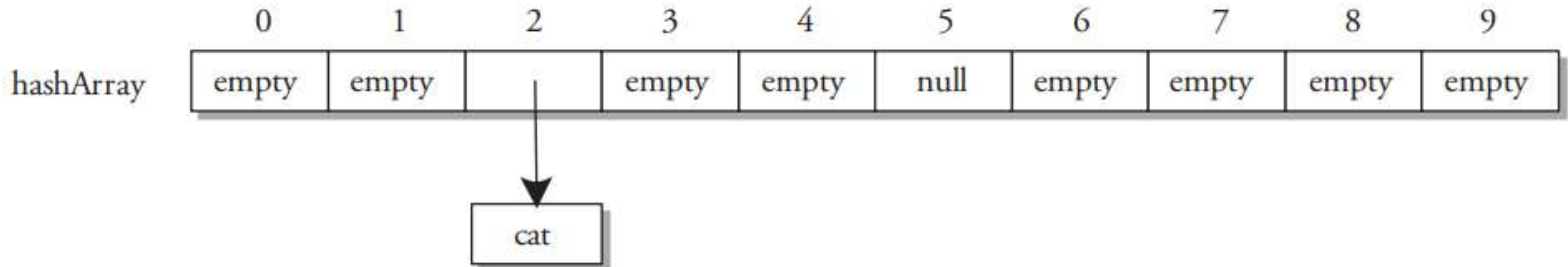
- ▶ Hash map is a data structure that efficiently stores and retrieves data from memory.
- ▶ An object is stored in a hash table by associating it with a *key*. Given the key, we can retrieve the object. Ideally, the key is unique to each object.
- ▶ If the object has no intrinsically unique key, then we can use a **hash function** to compute one. In most cases, the hash function computes a number.

Hash Map

- hashArray = new LinkedList [SIZE]; // *SIZE* = 10

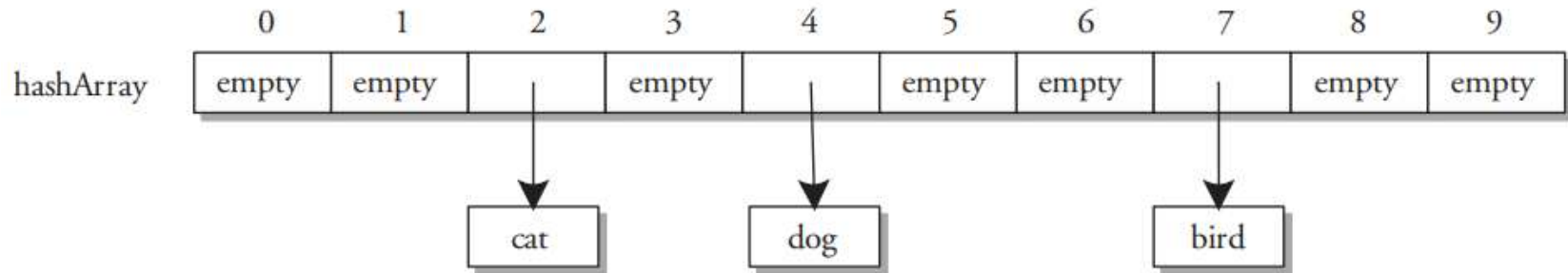


After adding “cat” with hash of 2



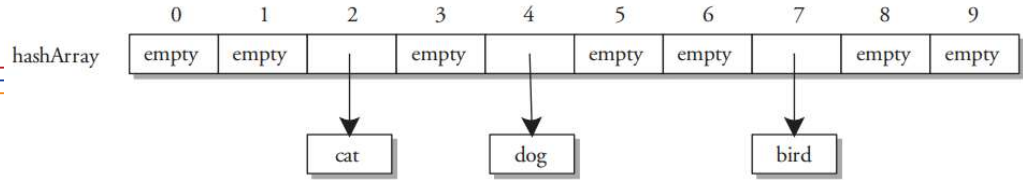
Hash Map

After adding “dog” with hash of 4 and “bird” with hash of 7

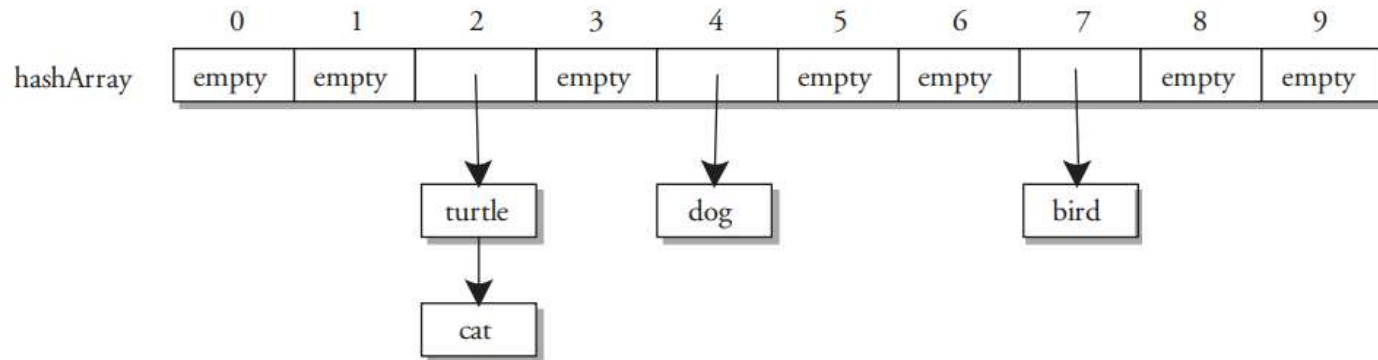


Hash Map

After adding “dog” with hash of 4 and “bird” with hash of 7



After adding “turtle” with hash of 2 – collision and chained to linked list with “cat”



Questions



Judge a man by his questions rather than by his answers.